

Checkpoint 2 — Aluno 2

Papel na arquitetura

O aluno 2 implementa o índice que permite descobrir um documento e localizar os Peers que possuem cada chunk. A identidade e a tabela de membros do checkpoint 1 continuam em uso; no checkpoint 2 entram ObjectID, FileMetadata, hash table, descritores, disponibilidade de chunks e integração de ANNOUNCE/LOOKUP no Super Peer. Os bytes do PDF permanecem nos Peers.

Arquivos principais

- node.c: valida IP, porta e UUID e calcula o NodeID SHA-256 usado também no checkpoint 2.
- superpeer.c: tabela de membros, JOIN/LEAVE, atendimento TCP de ANNOUNCE/LOOKUP e main. Incorpora os antigos membership.c e superpeer_app.c.
- metadata.c e metadata.h: ObjectID, FileMetadata, hash table e disponibilidades dos chunks.
- directory.c: converte os dados de transferência para o índice e transforma NodeIDs em IP/porta nas respostas de consulta.

Estrutura exigida no trabalho

```
typedef struct {
    uint8_t object_id[32];
    char filename[256];
    uint64_t size;
    uint32_t chunk_count;
    uint8_t **chunk_hashes;
    uint32_t version;
    uint32_t owner;
} FileMetadata;
```

object_id é SHA-256 dos bytes completos do arquivo; filename e size descrevem o PDF; chunk_count conta blocos de até 4 MiB; chunk_hashes guarda os hashes SHA-256 individuais; version começa em 1. owner é um resumo numérico dos quatro primeiros bytes do NodeID em ordem big-endian. Como esse resumo pode colidir, a localização usa o NodeID completo, guardado separadamente. LZ4 pertence ao documento de transferência, não a FileMetadata.

Fluxo das informações

1. O Peer faz JOIN. O Super Peer recalcula o NodeID recebido e o registra na tabela de membros se os dados forem consistentes.
2. Depois de confirmar o armazenamento, o Peer envia ANNOUNCE com ObjectID, nome, tamanho, descritores e hashes.
3. superpeer.c verifica se o NodeID de origem está cadastrado. directory.c converte o anúncio para FileMetadata e chama metadata_announce.
4. metadata.c valida os descritores, prepara uma entrada completa e publica o registro na hash table. Para cada chunk, associa o índice ao NodeID completo do anunciante.
5. Outro Peer faz LOOKUP por nome ou ObjectID. O índice devolve os descritores; directory.c consulta a tabela de membros e fornece IP/porta dos Peers disponíveis.
6. O download dos bytes é direto entre Peers. Em LEAVE voluntário, o Super Peer remove o membro e as localizações associadas.

Por que essas escolhas?

- ObjectID por SHA-256 incremental: identifica conteúdo independentemente do nome e não exige carregar o arquivo todo na memória. A implementação usa OpenSSL EVP.
- Hash table com 257 buckets: oferece índice local simples. Colisões de bucket usam encadeamento e comparação dos 32 bytes completos do ObjectID; a tabela não redimensiona.
- Registro esparsos: criar um documento não reserva antecipadamente um vetor para todos os chunks de um arquivo grande.
- Anúncio atômico: valida e aloca a nova entrada antes da troca, para que uma consulta não veja um documento publicado pela metade.
- Mutex: protege a hash table e a tabela de membros quando várias conexões são atendidas ao mesmo tempo.
- Cópias nas consultas: `metadata_find_document` devolve hashes independentes; o chamador usa `file_metadata_free`. Isso evita entregar ponteiros internos que podem perder validade.
- Estrutura local separada do wire: ponteiros C não são transmitidos; `transfer_protocol.c` serializa bytes e inteiros em ordem definida.

Demonstração e testes

No upload, a CLI apresenta File, Size, ObjectID, Chunks, hashes de cada chunk e Compression: LZ4. No download, mostra Download completed e SHA-256 verified. Essas mensagens resultam do fluxo integrado dos dois alunos, além da API local de metadados.

```
make -B all
make test-aluno2
make test-c2
python3 tests/c2/integration.py --bin-dir bin
bash script_testes_c2.sh
```

O teste local de membros compila `superpeer.c` com `SUPERPEER_MEMBERSHIP_ONLY` para excluir o atendimento TCP e o main. Na verificação desta versão, os testes locais e C2 passaram, a integração TCP aprovou 46 verificações e o roteiro C2 passou 20/20. API local e integração de rede são verificações distintas.

Limites da entrega

O índice do Super Peer fica em memória; não há persistência ou réplica distribuída desse índice. Reiniciar somente o Super Peer não pede automaticamente reanúncio aos Peers ativos. ALIVE e last_seen não implementam heartbeat ou detecção automática de falha. LEAVE cobre saída voluntária. O caminho de CRC inválido encerra a conexão sem a resposta ERROR exigida em parte do requisito. Chord, Gossip, SMR, 2PC e IST pertencem a checkpoints futuros.